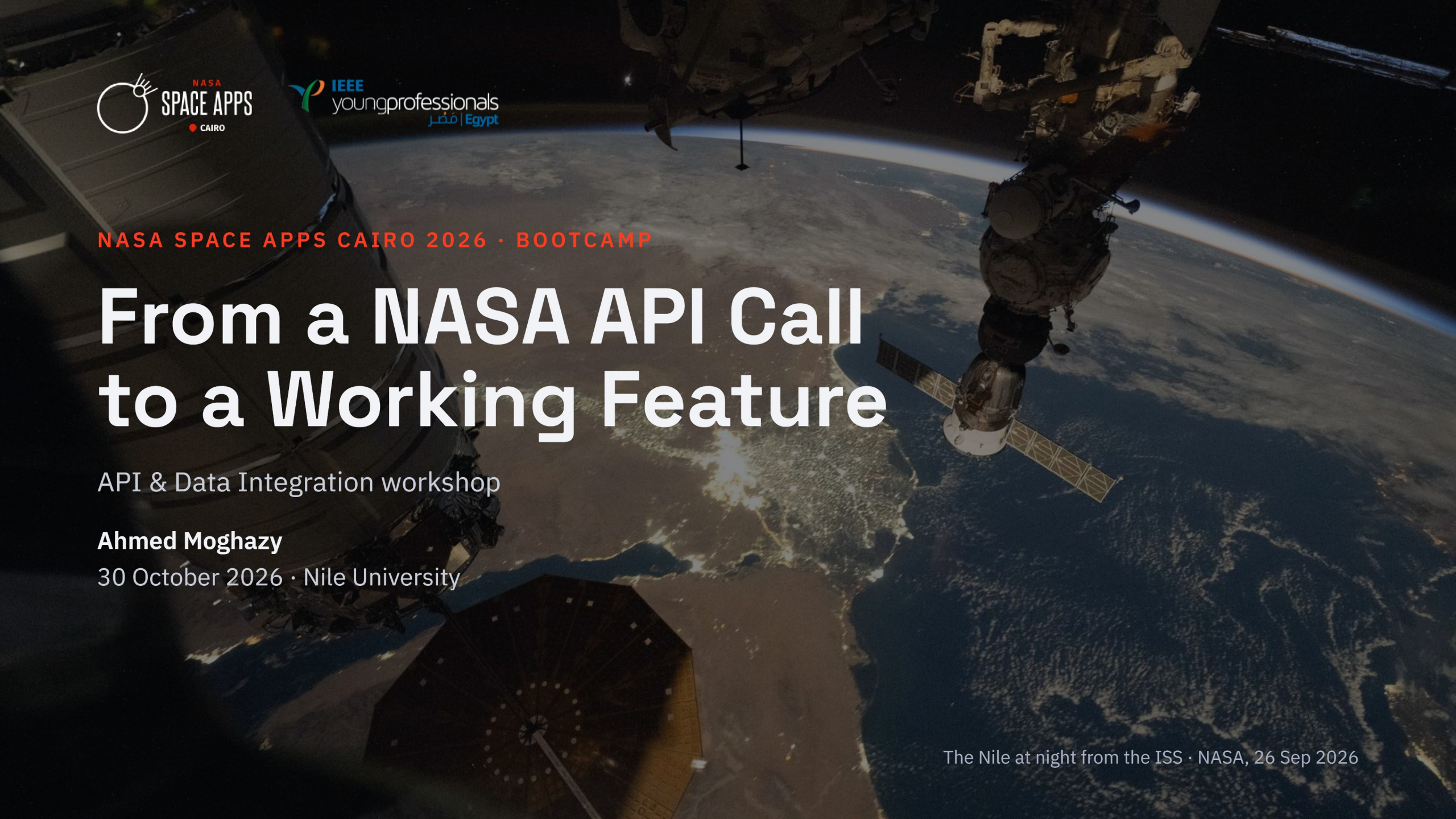


NASA SPACE APPS CAIRO 2026 · BOOTCAMP

From a NASA API Call to a Working Feature

API & Data Integration workshop

Ahmed Moghazy

30 October 2026 · Nile University

The Nile at night from the ISS · NASA, 26 Sep 2026

Why real NASA data comes first

Relevance

Judges want NASA data at the core of your project, not added on

Global Award

Only projects that use NASA data or resources can win one

2–3 hours

Get your first real call working and saved, then build

1 **Get real data flowing** → 2 **Build one feature** → 3 **Tell the story**

Today every pair leaves with: **a working call · a saved data file · one feature on screen**

Our two hours

0′	5′	30′	40′	50′	75′	110′
Welcome	Building blocks	Winners	Join & fail	Code-along	Hands-on build	Plan & close

By the end you can:

Explain the request → response cycle with a NASA example

Choose the right NASA source for your need, from its docs and limits

Build a clean table, a saved file and one working feature

Recover from 403s, 429s, empty results and missing values

NASA SPACE APPS CAIRO

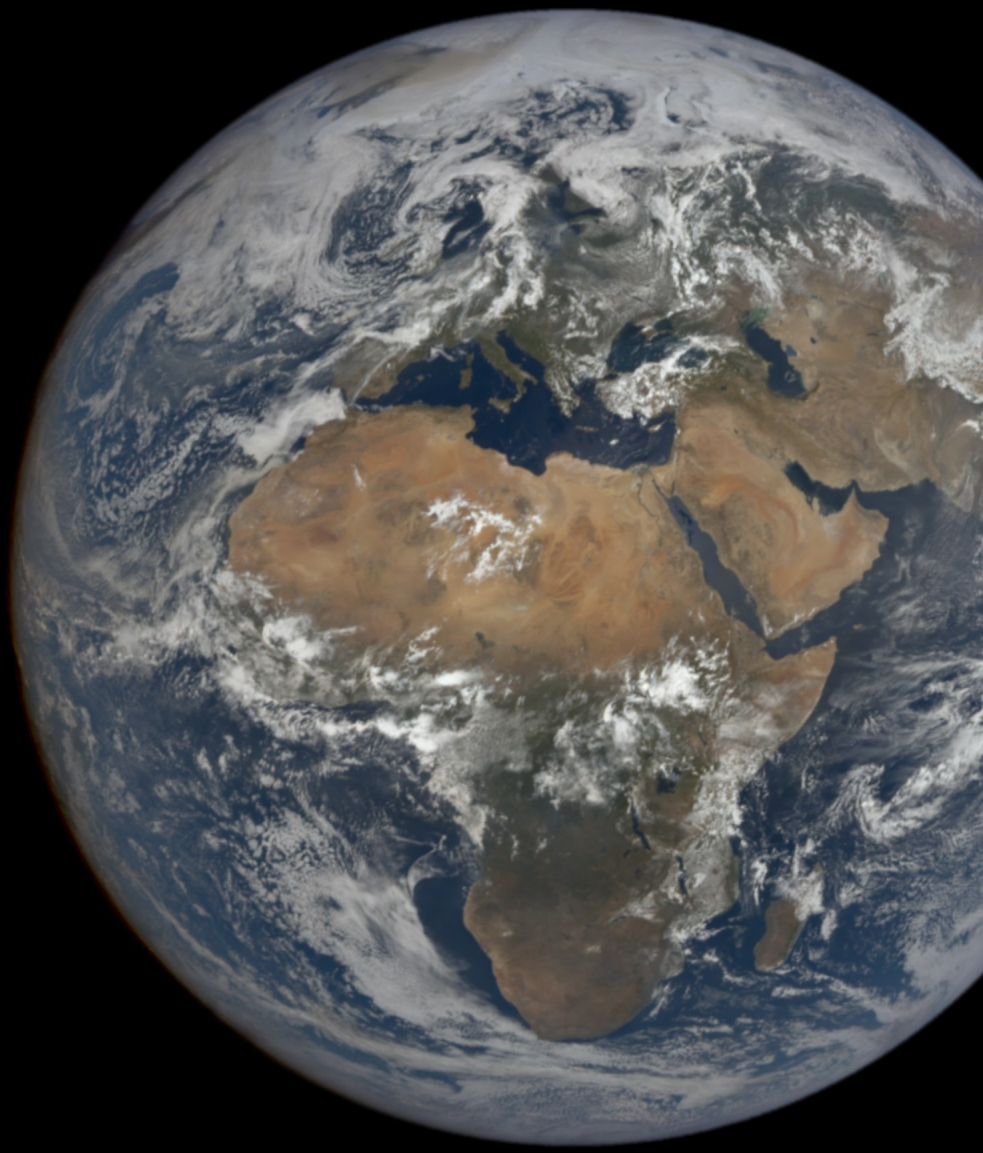
An API is a web address that returns data

Like opening a web page, but you get raw data (JSON) back instead of a website.

```
https://api.nasa.gov/neo/rest/v1/feed
?start_date=2025-10-01&end_date=2025-10-07
&api_key=YOUR_KEY
```

● **Endpoint** where ● **Parameters** which data ● **Key** who's asking

200 OK `{ "element_count": 37, … }`



Earth from NASA EPIC · 1 Jun 2025

NASA SPACE APPS CAIRO

The journey of one API call

Client

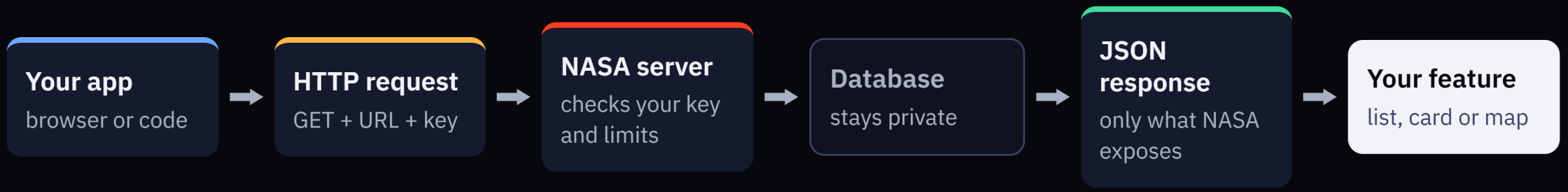
The one who asks: your browser, notebook or app.

Server

The one who answers: NASA's computers, which hold the data.

Request lifecycle

Ask → check the key → fetch → reply with JSON.



Every API you will ever use follows **this same cycle**.

Why NASA gives you an API, not its database

Problem	What the API does
Database too complex	Shows a simple menu of endpoints
Security	Keeps the raw tables private
Standardization	Everyone gets the same JSON shape
Scalability	One front door that can be cached and limited
Access control	Keys tell NASA who is asking

IMAGINE, BEHIND THE SCENES

`neo_objects` `neo_orbits`

`neo_size` `neo_hazard`

You never touch these tables. You call:

`GET /neo/rest/v1/feed`

and get only what NASA chooses to show.

An API is the contract between two systems.

Anatomy of an HTTP request

GET read data | **POST** create | **PUT** update | **DELETE** remove

NASA data APIs are almost always **GET**, because we only read.

REQUEST

```
GET /planetary/apod
?api_key=YOUR_KEY
Host: api.nasa.gov
Accept: application/json
```

Method · endpoint · parameters (with the key) · headers

RESPONSE

```
200 OK
Content-Type: application/json
{ "title": "…", "date": "…",
"url": "…" }
```

Status code · headers · body (JSON)

Status codes tell you what went wrong

200

It worked

{"element_count": 37, ...}

400

Bad parameter

“The Feed date limit is only 7 Days”

401/404

Wrong address

NASA answers 401 for a wrong endpoint

403

Bad or missing key

“API_KEY_INVALID”

429

Too many calls

DEMO_KEY allows 10 calls per hour

500/503

Their problem

Wait, retry, use your saved copy

Quiet failure: 200 OK with an empty list [ ] means no error, but also no data. Check for it.

Your key, and where it may live

10,000

calls per hour with your own free key

10

calls per hour on DEMO_KEY, shared by everyone on this Wi-Fi

Free NASA key in a web page: OK
Worst case, someone uses up your quota

Paid key (e.g. an AI key): never
Not in a web page, not in a public repo

Saved files can leak your key
NeoWs copies it into every link it returns

Browser or server?
A call from the page shows your key to anyone who looks

NASA SPACE APPS CAIRO

Reading API docs: seven things to find

01

Base URL

Where the API lives, e.g. api.nasa.gov

02

Endpoint

The path for one kind of data

03

Parameters

What you can ask for, and in which format

04

Response example

The JSON shape you will get back

05

Rate limits

How many calls per hour or day

06

Authentication

Is a key needed, and where does it go?

07

Error codes

What each failure looks like

Test the URL in your browser first

NASA SPACE APPS CAIRO

Docs in practice: the NeoWs Feed

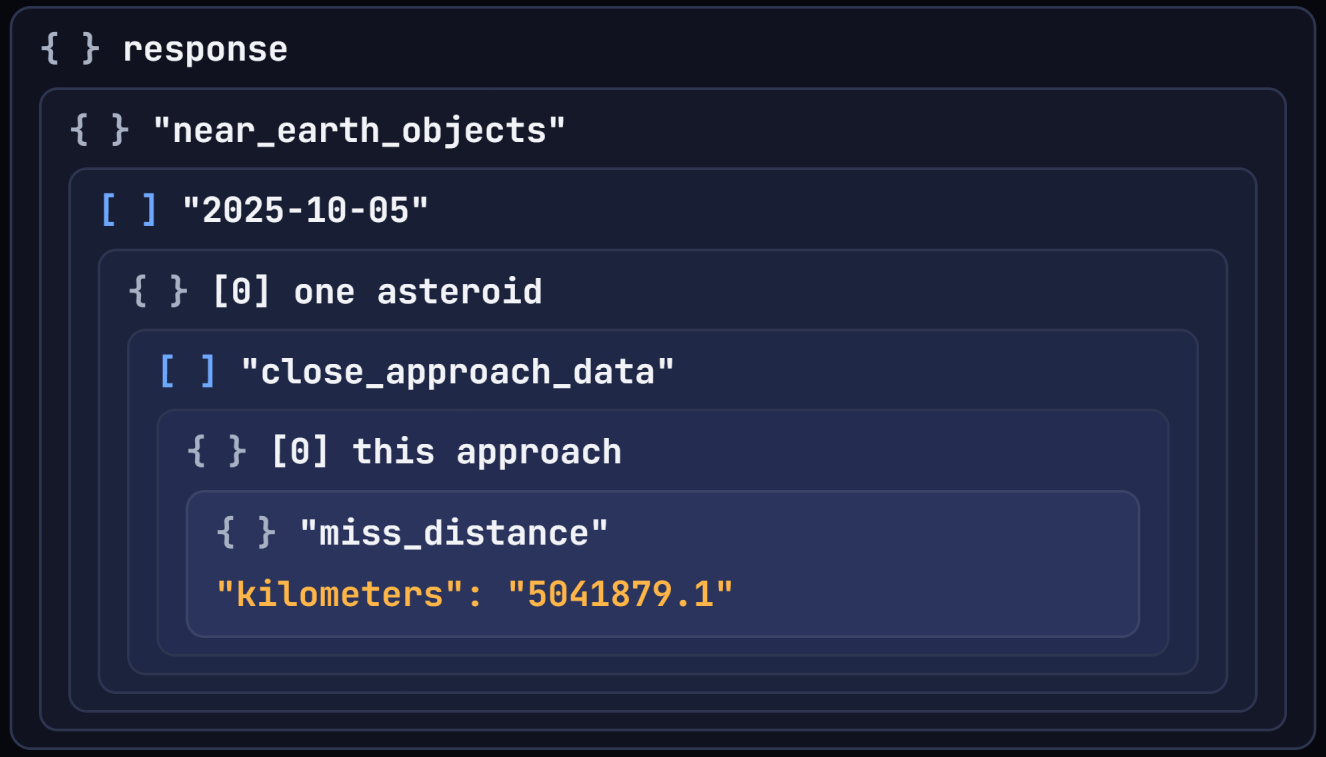
WHAT THE DOCS SAY

`GET https://api.nasa.gov/neo/rest/v1/feed`

Parameter	Type	Meaning
start_date	YYYY-MM-DD	first day
end_date	YYYY-MM-DD	default: start + 7 days
api_key	string	your key

Look for	In NeoWs
Base URL	api.nasa.gov
Endpoint	/neo/rest/v1/feed
Parameters	start_date, end_date, api_key
Response	near_earth_objects, grouped by date
Rate limits	per key, see api.nasa.gov
Authentication	api_key in the URL
Error codes	400 if the range is over 7 days

JSON is boxes inside boxes



{ } dictionary: look up by **name**

[ ] list: look up by **position**

```
feed["near_earth_objects"]
["2025-10-05"][0]
["close_approach_data"][0]
["miss_distance"]
["kilometers"]
```

"5041879.1" is text, not a number. Convert it before you sort or compare.

NASA SPACE APPS CAIRO

IN THE NOTEBOOK · PART 1

From nested JSON to a clean row

1 **Walk the dates**

The feed groups asteroids by day. Loop over near_earth_objects: each key is a date, each value a list.

2 **Walk the asteroids for that day**

Inner loop. Open one asteroid first and print its keys. Where is the size? The speed?

3 **Open the boxes**

Size sits in a nested box. Speed and miss distance sit inside the first close approach.

4 **Append one flat row**

Put the five fields into one dictionary per asteroid, then hand the list to pandas.

THE 5 FIELDS WE KEEP

- name
- size (metres)
- speed (km/s)
- miss distance (km)
- hazardous flag

Hint: JSON hands you numbers as text. Convert before you compare.

Clean before you use: units, gaps and dates

Problem	What it looks like	Fix
Numbers as text	"5041879.1"	float() before you sort or compare
Wrong units	km vs miles	Check the docs, convert once
Missing values	-999 in POWER	Turn into NaN, then drop or fill
Odd times	USGS time in ms	Convert to a real date
Empty result	200 OK and [ ]	Check the length before you show it

LATER, IN YOUR BUILD

Find each problem in your own data, fix it in one line, and print the column to prove it.

Hints: astype, replace with NaN, to_datetime.

The NASA data universe

APOD
Astronomy picture of the day

free key

NeoWs
Near-Earth asteroids by date

free key

EPIC
Whole-Earth photos from space

free key

DONKI
Space-weather events: flares and storms

free key

POWER
Weather and solar history for any place

no key

JPL Small-Body DB
Size and orbit of one asteroid or comet

no key

Exoplanet Archive
Confirmed planets, queried like a table

no key

Earthdata
Big satellite datasets, mostly file downloads

free login

Start from your need, then pick the source. Always check each one’s docs and limits.

Which NASA source for which need?

API
Live, small, filtered. Today's focus

File download
Big, bulk datasets (e.g. Earthdata)

Ready tool
Explore first (e.g. NASA Worldview)

Need	Source	Key?	Watch out
Asteroids passing Earth this week	NeoWs	free	Max 7 days per call; numbers as text
Whole-Earth photo for a date	EPIC	free	Empty list on dates with no photos
Weather history for a place	NASA POWER	none	-999 means missing
Size and orbit of one asteroid	JPL Small-Body DB	none	Look up by name or ID
Earthquakes (partner data)	USGS	none	Time in ms; [lon, lat] order
Space picture of the day	APOD	free	Placeholder data this week

How to dissect a winning project

01
Problem
What need were they solving?

02
Data sources
Which APIs and files did they use?

03
Integration point
Which shared field joins the sources?

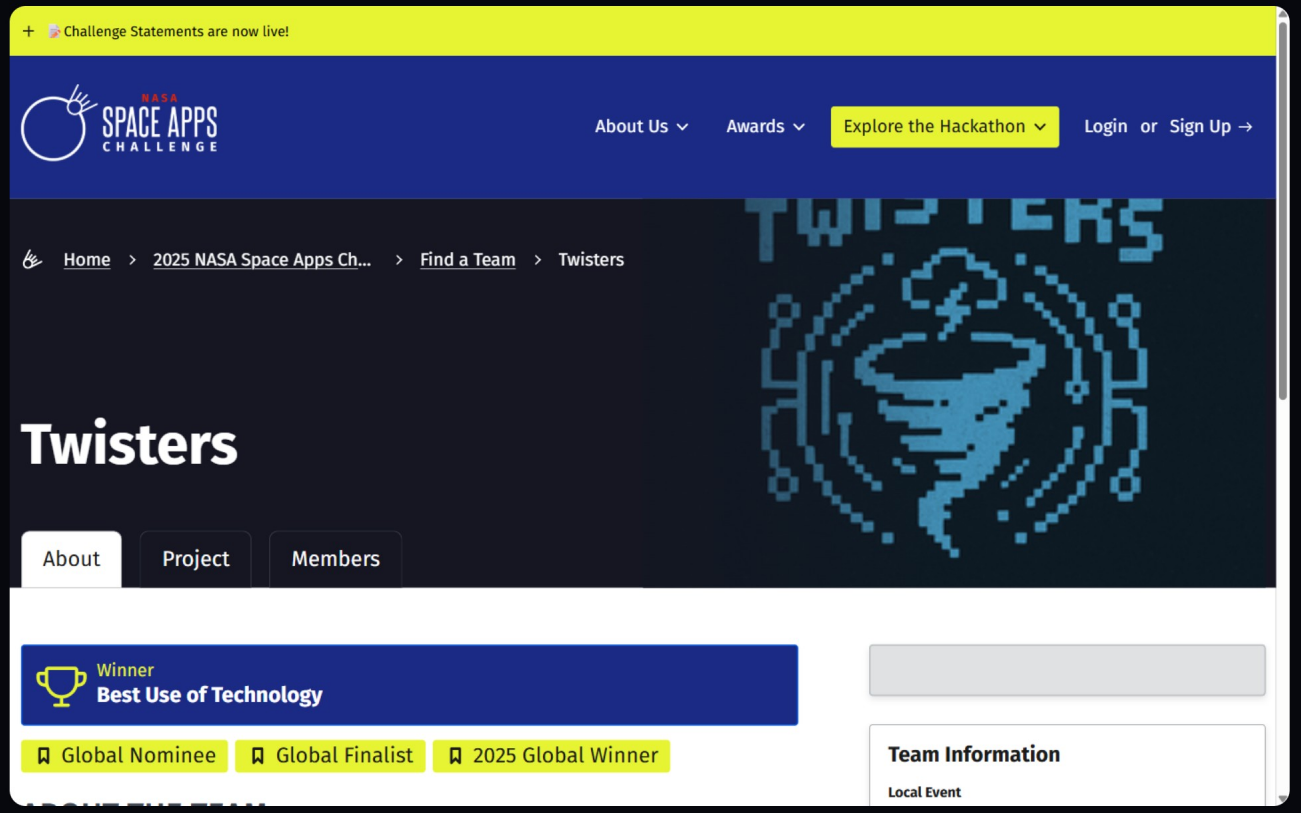
04
Engineering risks
What could fail on demo day?

05
Fallbacks
What keeps the demo alive?

06
Why it stands out
What makes judges remember it?

We use these six questions on three 2025 projects: **SkySense, Meteor Shield and TechNova.**

SkySense: many sources, one score



2025 · GLOBAL WINNER

Will It Rain On My Parade? · Twisters (Mexico) · Best Use of Technology

DATA SOURCES

NASA POWER history, a live forecast, Google and Gemini AI

INTEGRATION POINT

Place (lat, lon) and date. That shared field is the join.

ASK YOURSELF

What happens if one of four sources is slow or down?

WHY IT STANDS OUT

Many sources, one weather-risk score a user understands

Their live site is gone: the domain expired.

spaceappschallenge.org/2025/find-a-team/twisters

Meteor Shield: call, check, reshape, fall back

```
API_KEY = "SWu1••••••••••••••••"

const res = await fetch(`${API_BASE}/feed?
start_date=…&end_date=…&api_key=${API_KEY}`)
if (!res.ok) return getFallbackNEOData()
const data = await res.json()
if (!data?.near_earth_objects)
return getFallbackNEOData()
Object.values(data.near_earth_objects)
.forEach(day ⇒ all.push(...day))
return all
} catch { return getFallbackNEOData() }
```

2025 · HONORABLE MENTION

Meteor Madness · Team Polaris (Bangladesh)

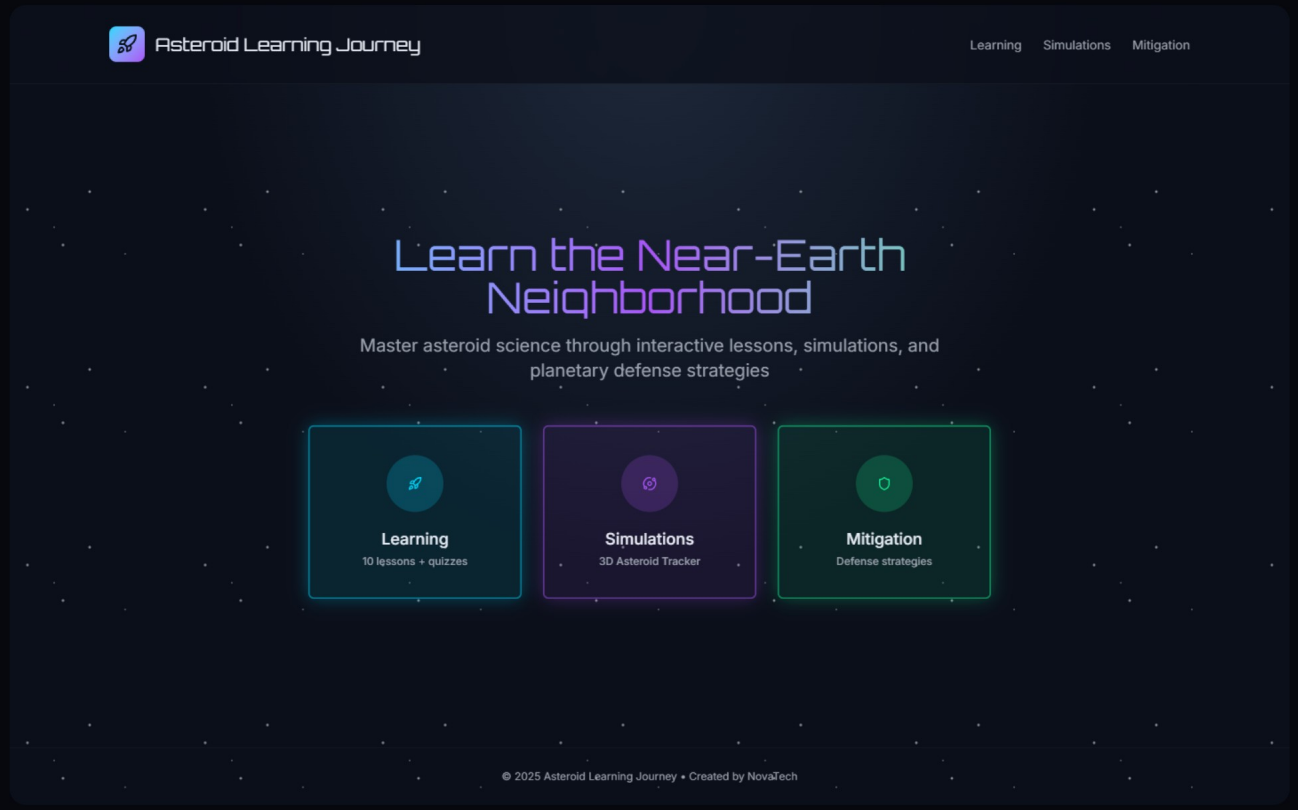
call check reshape show fallback

Integration point: merges the by-date lists into one list.

The key sits in page code. OK for a free NASA key, never for a paid one.

Every failure path returns built-in data, so the demo never shows a blank screen.

TechNova: same API, different feature



2025 · GLOBAL FINALIST

Meteor Madness · TechNova (Morocco)

How they built it

Learn → Simulate → Mitigate: NeoWs asteroids for a date range, then an impact on a 3D view and a map

Look for

The API gives you the data. Your idea makes the feature.

Browser or server: where does the call run, and why does it matter for keys?

NASA SPACE APPS CAIRO

spaceappschallenge.org/2025/find-a-team/technova3

Three patterns to copy

SKYSENSE

Join on one shared field

Place and date tie four sources into one score.

METEOR SHIELD

Fail gracefully

Every failure path returns built-in data, so the demo never goes blank.

TECHNOVA

Same data, new idea

One NeoWs feed became a Learn, Simulate and Mitigate app.

In all three, **NASA data sits at the core of the project**, not added on. That is what the Relevance score looks for.

AUDIENCE CHALLENGE · 3 MIN, IN PAIRS

You’re building a meteor-risk platform.

01 Which NASA APIs would you choose?

02 Where do they connect? Name the shared field.

03 What happens when one of them fails?

A 2025-style scenario. Talk in pairs, then share one answer.

Meteor risk: who passes by, and who might hit

SOURCE 1 · NASA NEOWS

The traffic report

Which asteroids passed near Earth this week: size, speed, miss distance, a hazardous flag and is_sentry_object.

```
GET api.nasa.gov/neo/rest/v1/feed
?start_date=…&end_date=…&api_key=…
```

Free key · max 7 days · api.nasa.gov → Asteroids NeoWs

SOURCE 2 · NASA JPL SENTRY

The watch list

Asteroids with any chance of hitting Earth in the next ~100 years: impact probability, dates and a Torino score (0–10).

```
GET ssd-api.jpl.nasa.gov/sentry.api
?des=(one asteroid)
```

No key · ssd-api.jpl.nasa.gov/doc/sentry.html

join on: asteroid ID NeoWs says who is around. Sentry says how risky they are.

NASA SPACE APPS CAIRO

Integration = joining by a shared field

SOURCE A · NASA NEOWS

```
{ "name": "2025 AB",
"date": "2025-10-05" }
```

join on: date

SOURCE B · USGS EARTHQUAKES

```
{ "date": "2025-10-05",
"magnitude": 5.8 }
```

RESULT · ASTEROID + EARTHQUAKE CONTEXT

2025 AB · 2025-10-05 · magnitude 5.8 the same day

Illustrative values. Words to know: **join** · **key** · **relationship** · **data fusion** (many sources, one picture).

IN THE NOTEBOOK · PART 3

Joining in pandas, and where joins go wrong

1 **Make both dates the same type**
USGS time is milliseconds since 1970. NeoWs date is text. Turn both into plain dates.

2 **Reduce quakes to one row per day**
Group by date and keep the strongest magnitude. Otherwise rows multiply.

3 **Merge on the shared date**
Keep every asteroid, even on days with no quake. Which how= does that?

date	name	mag
2025-10-05	2025 AB	5.8
2025-10-06	2025 CD	NaN

Illustrative target shape, not real data.

WHERE JOINS GO WRONG

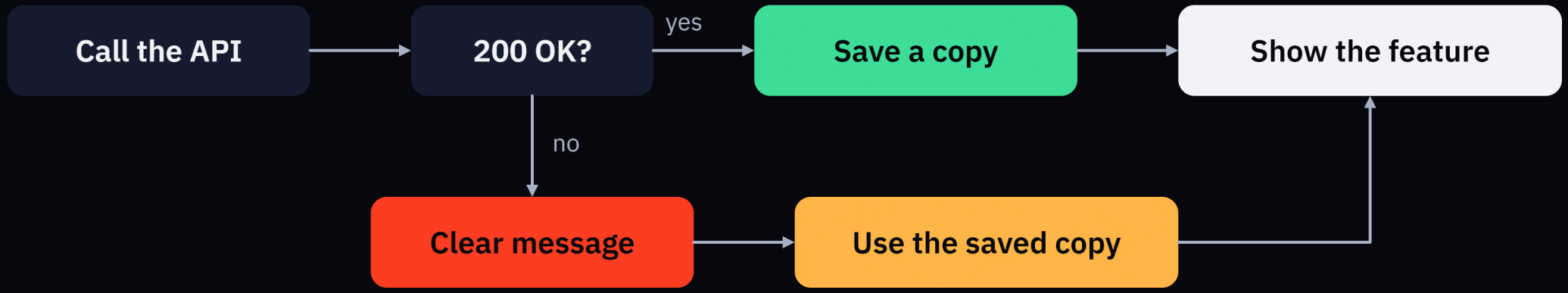
- Same format on both sides: dates as dates, not text on one side and ms on the other
- Time zones: UTC vs local time can shift the day
- GeoJSON order is [lon, lat], not [lat, lon]
- One-to-many: group first, or rows multiply
- A left join keeps every asteroid, even on days with no quake

Failure engineering: what breaks, and what to do

Failure	What it looks like	What to do
Timeout	NASA is slow	Set timeout=10, retry once, then fall back
403	Invalid or missing key	Show a clear message; check the key
429	Too many calls	Wait, cache results, use your own key
Missing field	A key is not in the JSON	Use .get() with a default value
Wrong units	km vs miles	Check the docs; convert once
Empty response	200 OK with [ ]	Show a “no data” state, not a blank screen

Real systems survive because they plan for failure.

Plan for failure: things broke this week



APOD API

Returns a placeholder (“NASA Science”) for every date, and 500 for today

SkySense

A 2025 Global Winner. The live site is gone: the domain expired

Meteor Shield

Fallback data built in, so it still shows something when calls fail

IN THE NOTEBOOK · PART 1, PLANNED FAILURE

The fallback pattern: build it yourself

1 **Ask before you trust**
Call with a timeout. Check the status code. Then check the answer is not empty.

2 **Save when it works**
Every good answer goes to a backup file. Hide the key before you save.

3 **Catch when it fails**
Wrap the call in try / except. Print one clear message a teammate can read.

4 **Fall back, in order**
Live call, then the saved copy, then clearly labelled sample data.

PLANNED FAILURE

The notebook switches on a wrong key. Your function must survive it and still show a result.

Test it: break the key on purpose, run, and read your own message.

Add an AI? Free providers

Optional. Same idea as every API today: one POST, one key, one JSON answer. Keys live in Colab Secrets, never in code.

GOOGLE GEMINI

AI Studio key

Free tier on Flash models

Free prompts may be used by Google: no private data

GROQ

Very fast

Free plan, limits per organisation

Check the dashboard: limits change

OPENROUTER

Router + :free models

One key, many free models

A paid model in your list costs money

All three accept the same request:

```
POST …/chat/completions Authorization: Bearer KEY
```

NASA SPACE APPS CAIRO

Details: LLM cheat sheet (in the repo)

What comes back from an AI call

Your code

POST /chat/completions

Key in the header. Messages in the body.

AI provider

ANSWER COMES BACK AS JSON

`choices[0].message.content · finish_reason · usage.total_tokens`

The three fields you read: the text, why it stopped (stop, or length = cut off), what it cost

TEXT

Plain answer

Read the content string. It may be empty or cut off: check finish_reason.

JSON

Structured answer

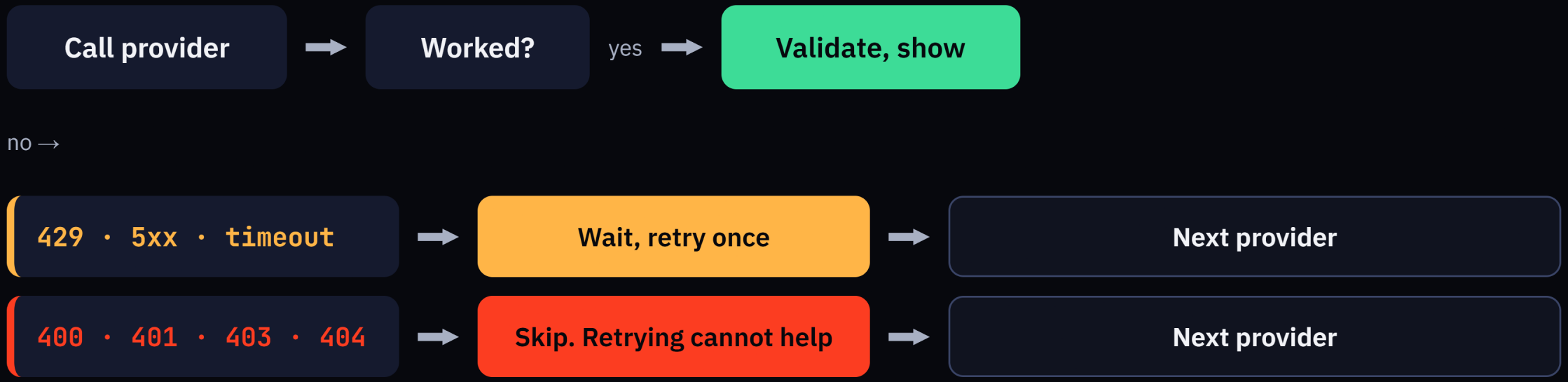
JSON mode is only a hint. Strip code fences, parse, validate keys, retry once.

STREAM

Words as they come

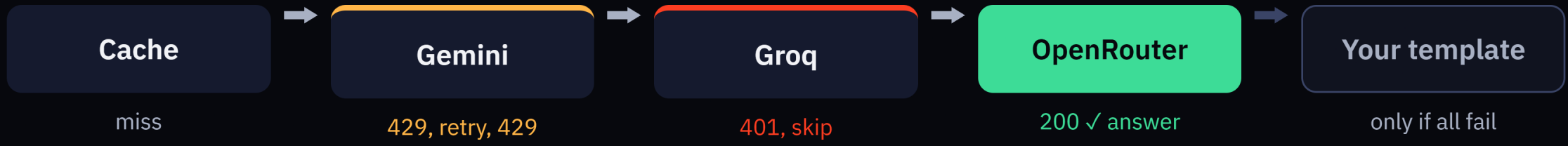
Lines start with data: and end with [DONE]. Show text while it arrives.

When the AI call fails



Rule of thumb: temporary problem? Try again once. Permanent problem? Move on.

The fallback chain, step by step



The user always sees something. The screen says who wrote it: “AI-written by OpenRouter”, or “Basic version (no AI)”.

Good answers go into the cache, so a repeat question costs nothing and still works if every provider is down later.

AI responses and the fallback

TEXT · JSON · STREAM

Know what you asked for

choices[0].message.content; check finish_reason

JSON mode is a hint: parse and validate

WHEN IT FAILS

Retry or skip?

429 and 5xx: wait, retry once. 401, 403, 404: skip

Never loop forever on a bad key

THE CHAIN

Always show something

Cache, provider 1, 2, 3, then your template

Say on screen when it is the basic version

Facts come from your code. The model only phrases them. Check its numbers before you show them.

Architecture: start small, join later



Get **one source working end to end** first. Add the second source only when the first feature runs on saved data.

COMMON MISTAKES

- Many APIs at once
- Your own backend or database
- Huge downloads
- A “perfect” data pipeline

NASA SPACE APPS CAIRO

Code-along: from the docs to a feature

01 Need

02 Find the API

03 Read the docs

04 Test URL in browser

05 Code the call

06 Inspect JSON

07 Pick fields

08 Clean

09 Save a backup

10 Build the feature

11 Handle errors

wrong key on purpose → message → saved copy

min 85
200 OK

min 95
clean table

min 108
feature + saved file

Open the notebook in Colab

File → Save a copy in Drive

github.com/moghazy17/
nsac-2026-api-workshop

NASA SPACE APPS CAIRO

Pick one API card with your partner

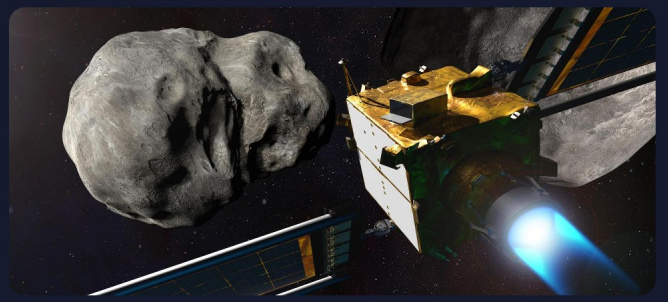


EASY · FREE KEY

EPIC: Earth from space

Pick a date, list the day’s photos, show an Earth card

Trap: an empty list on dates with no photos



MEDIUM · FREE KEY

NeoWs: asteroid list

Your twist: only hazardous, the biggest, or per day

Trap: numbers arrive as text; 7-day limit



HARDER · NO KEY

POWER: a very hot day?

20 years of Cairo data → % of days above 35 °C

Trap: -999 means missing

NASA SPACE APPS CAIRO

Photos: NASA EPIC · NASA/Johns Hopkins APL (DART, illustration) · ISS crew

Your 48-hour data plan

HOURS 0–3

One real call working, response saved

→

NEXT

Build the feature on the saved copy

→

LATER

Go live; join a second source by date or place

Don’t waste time on

- Your own backend or database
- Many APIs at once
- Huge downloads
- A “perfect” data pipeline

AI assistants: useful, but check them

- Ask them to explain docs and draft parsing code
- Check every field against the real response: an AI gave us danger_level and diameter_km, which don’t exist
- Never paste a paid key into code you share

5 things to remember

1. Test the API URL in your browser first
2. Get your own free key; a paid key never goes public
3. Save a copy the moment your call works
4. Take only the fields you need; check units and -999
5. Plan for failure: a clear message + your saved data

Thank you! Notebook, API cards and cheat sheet:
github.com/moghazy17/nsac-2026-api-workshop

NASA SPACE APPS CAIRO



Cairo at night from the ISS · NASA, Jul 2026